THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Units: 1 – 5

Chosen Software: Git | License: GNU GPL v2

Student Name: Hardik Gaur

Registration Number: 24BAI10484

Slot: A13

Date of Submission: 31 March 2026

Part A — Origin and Philosophy

A1 — The Problem Git Was Created to Solve

The act of software development has always been a collaborative one; because when you have hundreds (if not thousands) of programmers around the globe working on the same codebase at the same time adding features, fixing bugs, and submitting improvements, there needs to be a system in place to track each individual change and version of that code, along with any potential conflicts that may arise. This is what version control systems do. But during the 1990s, version control was still considered to be a solved problem in theory only.
The Linux kernel was created by Linus Torvalds in 1991. While it originally started off as just a personal project, it has since grown into one of the largest and most successful open-source software projects in history, with numerous individuals contributing to its development from various locations around the world via email. For the first 10 years of the Linux kernel's existence, Linus handled everything manually. He had each developer create a patch file that he would receive via email, then he would review all of those files before merging them directly into the main source code tree often requiring him to manually apply the patches by hand. This made Linus essentially a human version control system for the first decade of Linux development.

Everyone recognized that this was not feasible. Before Git, most software development relied on tools like CVS (Concurrent Versions System) and later Subversion (SVN), designed primarily for smaller, centrally managed projects. This led to multiple developers accessing a single server, creating bottlenecks and single points of failure. In addition, Linus had established that every task done daily must complete in under three seconds. Both CVS and SVN fell far short of that requirement for a project the size of the Linux kernel.

Linus's decision to use BitKeeper a proprietary, closed-source version control system built by a company called BitMover as the tool to manage Linux kernel development was possibly the biggest shock to the open-source community in 2002. It was deeply uncomfortable for the community to discover that the creator of the world's largest open-source project was now using a proprietary tool whose source code no one could read, modify, or redistribute. BitMover did offer the Linux community a free-of-charge license, but it came with significant restrictions: developers using BitKeeper were not allowed to work on competing version control projects, and BitMover retained control over certain project metadata.

Linus's rationale was based on practical application rather than ideology. BitKeeper was simply the best tool available reliable, fast, and capable of handling the scale and complexity of kernel development. The benefits in terms of productivity were significantly greater than the philosophical discomfort of using it. This uneasy arrangement lasted for three years. Then, in early 2005, it collapsed. Andrew Tridgell, a well-known kernel developer and creator of Samba, an open-source networking tool began analysing BitKeeper's network protocol and developed an application called SourcePuller to pull data from BitKeeper repositories using this protocol. BitMover interpreted this as reverse engineering of their proprietary software, which violated the license agreement. They revoked the free license for the entire Linux Kernel Project. Overnight, thousands of developers lost access to the tool their entire workflow depended on.

Linus was now in a genuine crisis. He could not use BitKeeper. CVS and Subversion were inadequate. He tried the few newer distributed systems that existed — Monotone and GNU Arch but they were either too slow or too complex to process the volume of patches the kernel required at the necessary speed. He evaluated every available option and found all of them wanting. So, Linus built an alternative version himself.

Linus Torvalds started working on creating Git on April 3, 2005. He announced the project publicly via a mailing list on April 6, 2005. By April 7, Git was already self-hosting meaning Linus was using Git itself to track Git's own development. This is considered an extremely important milestone in software engineering: it means Git was sufficiently functional to manage its own source code within four days of the first line being written. Git was benchmarked at 6.7 patches per second as of April 29, 2005. By June 2005, it was managing the official Linux Kernel releases.

The name Git was chosen with characteristic irreverence. Linus felt it could have multiple interpretations a random three-letter pronounceable word, a mispronunciation of 'get', or simply British slang for a stupid or unworthy person, which he jokingly applied to himself. His intent was not to build something polished for the masses; he simply wanted to provide a quick, effective solution to an urgent problem, with no intention of making it a permanent product.

Linus had, at no time, planned to maintain Git permanently. Within a week of the first commit, Junio Hamano became actively involved in Git's development. Linus recognised his talent and loyalty, and on July 26, 2005 less than four months after development began he transferred the entire repository to Junio Hamano as its active maintainer. Hamano released Git 1.0, the first stable version, on December 21, 2005, and has maintained Git ever since.

While there are technical aspects at play, the philosophical element of Git's creation is just as significant if not more so. Git was born not from ambition or commercial opportunity, but from the failure of a proprietary dependency. When a private firm decided to remove access to its tool, an entire open-source community had its hands tied because they had trusted something they could not control. Linus responded by building a replacement that was fast, decentralised, and fully free something no company could ever revoke. The vast majority of developers globally now use Git, all as a direct consequence of a licensing dispute that lasted less than one month in 2005. That is the epitome of what open source represents.

A2 — The License: What It Actually Says

To better understand Git, it is important to first know that Git is available under the GNU General Public License Version 2 commonly written as GPL v2. The GPL v2 is not just a legal formality; it is the mechanism by which Linus Torvalds ensured that no one could ever do to Git what BitMover had done to the Linux community. It is the legal foundation for the four freedoms.

What are the four freedoms, and does Git grant all of them?

The concept of the four freedoms was defined by Richard Stallman, founder of the Free Software Foundation, as the minimum set of rights that any user must have for software to be considered genuinely free. The term 'free' here does not refer to cost but to freedom like 'free speech' as opposed to 'free beer'. The four freedoms are:

Freedom 0 — the freedom to run the program as you wish, for any purpose.
Freedom 1 — the freedom to study how the program works and change it, which requires access to the source code.
Freedom 2 — the freedom to redistribute copies so you can help others.
Freedom 3 — the freedom to distribute copies of your modified versions to others, so the whole community can benefit.

The numbering starting at zero is itself a programmer's joke — and a deliberate philosophical statement. Freedom 0, the most basic right of simply being allowed to run the software, was added last when Stallman realised he had forgotten to state what should have been obvious. The fact that he had to add it tells you how aggressively proprietary software had, even by the 1980s, begun restricting this most fundamental freedom.

Does Git's GPL v2 license grant all four freedoms? Yes, completely and without restriction. Everyone on earth has the right to download Git's source code, read every line of it, run it on any machine for any purpose, modify it however they choose, and distribute those modifications to others. Git is free in both senses of the word — free of cost and free of restriction.

Can a company take Git, modify it, and sell it without sharing their changes?
This is where the GPL v2 becomes genuinely interesting. The short answer is no. At least not if they distribute that modified version to others. GPL v2 is a strong copyleft license: any modifications made to GPL v2-licensed code must also be released under GPL v2 when distributed. This is sometimes called the 'share-alike' principle and it is entirely intentional. The GPL v2 uses copyright law, the same mechanism that proprietary companies use to lock down code and turns it on its head to enforce sharing. This concept is called copyleft, one of the most clever legal inventions in software history.

A company can legally charge money for a modified version of Git. However, once they distribute that modified version to anyone outside their own organisation, they are legally required to also make the source code available under the same GPL v2 terms. They cannot keep improvements proprietary. One important nuance: the GPL v2 does not force redistribution at all. A company can fork Git, modify it extensively, and use it entirely internally without ever sharing those changes. The obligation to release source code only triggers at the point of distribution. This is how many large technology companies legally use GPL software internally.

What is the difference between GPL and MIT licensing?
The MIT License is short enough to read in under a minute. It essentially says: do whatever you want with this software copy it, modify it, sell it, incorporate it into a closed-source product as long as you include the original copyright notice. That is the only obligation. The GPL v2 imposes a significantly more important condition: any derivative work distributed to others must also be released under GPL v2, making source code available.

In practical terms: MIT says 'take this and do what you like.' GPL v2 says 'take this, do what you like, but if you give it to others, give them the same freedoms you received.' MIT is generous in the short term it allows proprietary companies to absorb open-

source work without giving anything back. GPL v2 is generous in the long term it ensures that every improvement made to the software, whoever makes it, eventually flows back to the commons.

If I were to build something and choose a license, the answer is a question of values rather than strategy. For a library where I wanted maximum adoption including by commercial developers I would choose MIT, as the lower barrier would spread the code further and faster. But for a tool I believed should remain permanently in the hands of its users, preventing any company from privatising improvements built on my work, I would choose GPL v2. Git's choice of GPL v2 reflects exactly this second philosophy. Linus had just watched a company revoke access to a tool the community depended on. He was not interested in making it easy for the next company to do the same to Git.

Free as in free beer vs free as in freedom

This is one of the most important distinctions in all of open-source philosophy, and Git illustrates it perfectly. 'Free as in free beer' means zero cost. You receive it at no charge. But like a free beer at an event it may come with conditions. The company providing it may be collecting your data, your attention, or your lock-in. You are not truly free; you are just not paying in cash.

BitKeeper was free as in free beer for the Linux kernel community. No money changed hands. But the community was not free they could not modify it, could not study it, and could not guarantee continued access. The moment BitMover decided the arrangement was no longer in their interest, the 'free' evaporated entirely.

'Free as in freedom' what the FSF and Stallman call libre means something far stronger. It means you have the legal, irrevocable right to use the software, study it, change it, and share it. No company can revoke that right. Git is free as in freedom. The GPL v2 guarantees this not through goodwill or corporate policy, but through law. Even if Junio Hamano stopped maintaining Git tomorrow, or if the entire community of contributors disappeared, every person on Earth would retain the legal right to take Git's source code and continue developing it. That is what freedom in software actually means, precisely the freedom that BitKeeper denied, and that Linus deliberately chose to restore.

A3 — The Ethics of Open Source

Ethics is not a comfortable topic in software development. Most engineers would rather debate algorithms than principles. But open source forces the question because it is, at its heart, a statement about how knowledge should be owned, who gets to profit from collective work, and whether a tool built by thousands of volunteers can survive contact with corporate capitalism. There are no correct answers here. What matters is honest thinking.

Should all software be open source? The case for.

The strongest argument for open source begins with a simple observation: most software that truly matters is infrastructure. It keeps hospitals running, powers financial systems, controls physical devices, and forms the backbone of the internet. When that software is proprietary, the public has no ability to audit it, verify it, or hold it

accountable. We are asked to trust companies whose primary obligation is to their shareholders, not their users.

Open source breaks this dependency. When source code is publicly available, security researchers can find and report vulnerabilities before malicious actors exploit them. Governments can verify that software used in elections or public services does not contain hidden backdoors. Independent developers can fix bugs without waiting for a vendor to prioritise them. The collective intelligence of thousands of developers, working transparently, consistently produces more reliable and more secure software than any single private team could build.

Git itself is a demonstration of this principle. Before Git, the Linux kernel's development depended on a proprietary tool that one company could and did revoke. The moment that happened, thousands of developers were helpless. Open source eliminates that vulnerability entirely. There is also an argument rooted in public investment: a significant proportion of foundational research from internet protocols to compilers to operating systems was funded by universities, governments, and public institutions. If taxpayers paid for the research that made a tool possible, it is difficult to argue that a private company has a moral right to lock it away and charge for access.

The case against why not all software should be open source.
The argument against mandatory open source is equally serious and deserves honest engagement. Building software is expensive. It requires skilled engineers who need to be paid, infrastructure that needs to be maintained, and years of sustained effort before the product becomes useful. Open source does not solve the funding problem it relocates it. Someone still has to pay the developers. When everyone is free to use the software without paying, the financial burden falls on a small number of contributors who are either employed by companies with a commercial interest in the project, or who donate their time voluntarily. This is not a stable model for critical infrastructure.

There is also a legitimate argument around competitive incentive. Proprietary software companies invest heavily in research and development precisely because they can recoup that investment through licensing revenue. If a company knows that any breakthrough must immediately be made public, the financial incentive to invest in that breakthrough diminishes significantly. Finally, not all software is infrastructure. A game, a creative tool, a niche application built for a specific commercial purpose these do not carry the same public interest argument. Requiring their source to be open is difficult to justify.

Is it ethical for a company to build profits on free community labour?
This is the most contested question in contemporary open-source ethics, and the most honest answer is: it depends entirely on what the company gives back.

Consider Red Hat. Founded in 1993, Red Hat built a multi-billion dollar business on Linux a free, open-source operating system that it packaged, supported, and sold to enterprises. The company was eventually acquired by IBM for approximately thirty-four billion dollars. On its face, this looks like a company extracting enormous value from unpaid volunteers. But the picture is more complicated. Red Hat employed many of the top open-source developers who contributed code to a wide variety of projects.

Their money flowed back into open-source development, making the entire ecosystem better.

But Red Hat also demonstrates where this arrangement can break down. In 2023, Red Hat restricted access to the source code of Red Hat Enterprise Linux, effectively ending the free downstream distributions that thousands of organisations depended on. This sparked one of the most significant ethical debates in the Linux community in years. Some described the decision as throwing a nuclear bomb into the Linux community. It felt like Red Hat had used open source to build its business, and then, once large and profitable enough, had begun to pull up the ladder behind it.

Google and Meta raise similar questions. Both run their global infrastructure on Linux and use hundreds of open-source tools. Both occasionally contribute back Meta has released React and PyTorch, Google contributes to Android and Chromium but the contributions are selective and strategic, not proportional to the value extracted. Whether this is ethical does not have a clean answer. It is legal. But 'legal' and 'right' are not the same thing.

My own position is this: a company that profits from open-source software and contributes back proportionally in code, funding, or engineering time is behaving ethically. A company that takes from the commons and gives nothing back is a free rider. Free riding is not illegal, but it is the slow erosion of the system that everyone depends on. The GPL v2 license that Git uses is specifically designed to prevent the most egregious form of this behaviour by legally requiring that improvements be shared but no license can fully solve a problem that is ultimately cultural, not contractual.

What does 'standing on the shoulders of giants' mean in software?
The phrase comes from Isaac Newton, who wrote in 1675 that if he had seen further than others, it was by standing on the shoulders of giants. In software, the giants are the layers of open-source infrastructure that every application proprietary or open is built on top of.

Git itself is a giant that others stand on. GitHub, acquired by Microsoft for seven and a half billion dollars, is built entirely on top of Git. GitLab, Bitbucket, every continuous integration system, every deployment pipeline all depend on Git as their foundation. None of that commercial value would exist without the ten days of work Linus did in April 2005 and the twenty years of maintenance Junio Hamano has provided since.

At the application layer, the picture is more complicated. A startup building on top of open-source infrastructure benefits from decades of free foundational work. But if that startup builds something genuinely new a new algorithm, a new user experience, a new way of solving a problem the open-source model does not require them to give that away. Innovation at the top of the stack can remain proprietary even when the foundations are open. Open source does not undermine original thinking it changes where original thinking is most needed. The giants of open source have already solved the infrastructure problems. What remains is to build something genuinely new on top of that foundation. That is not a constraint on innovation. It is the most efficient possible starting point for it.

Part B — Linux Footprint of Git

Every piece of software that runs on Linux leaves a footprint a set of files, directories, processes, and configurations that define how it lives on the system. Understanding that footprint is not just a technical exercise; it reveals how a tool integrates with the operating system, how it manages permissions and security, and how the open-source community keeps it updated. Git's Linux footprint is notably clean and minimal, which reflects the same philosophy that shaped its design: do one thing well and stay out of the way.

Installation Method

Git is available through the native package managers of every major Linux distribution. This is the first thing that distinguishes it from proprietary software, which typically requires downloading an installer from a vendor's website, accepting a licence agreement, and running an executable whose contents you cannot inspect. With Git, the installation is transparent, fast, and fully auditable.
On Debian-based systems such as Ubuntu the most common distribution for students and developers the installation command is:

```
sudo apt-get install git
```

On RPM-based systems such as Fedora, Red Hat, or CentOS, the equivalent command is:

```
sudo dnf install git
```

Both commands query the distribution's official package repository, download the Git package along with any dependencies, verify the package's cryptographic signature to confirm it has not been tampered with, and install everything. The process takes under a minute and requires no interaction beyond entering the command. On many Linux systems, Git is pre-installed by default.

It is also possible to compile Git from source, which requires tools like autotools, curl, zlib, openssl, expat, and libiconv. The fact that this is possible at all that anyone can download every line of Git's source code, read it, modify it, and build a custom version is a direct expression of the GPL v2 freedoms discussed in Part A2.

Key Directories Where Git Lives on the System

Once installed, Git places its files in standard Linux filesystem locations that follow the Filesystem Hierarchy Standard the convention that defines where different types of files should live on a Linux system.

The primary Git executable lives at /usr/bin/git. This is where the core Git commands reside the binary that runs when you type git at the terminal. Git installs a large number of sub-command executables in the same directory, each responsible for a specific operation: git-commit, git-merge, git-push, and so on.

Git's configuration is stored in three separate locations, each corresponding to a different scope of influence. The system-wide configuration file lives at /etc/gitconfig and applies to every user on the machine. The user-level global configuration file lives

at ~/.gitconfig in the individual user's home directory and applies to all repositories that user works with. Finally, each repository contains its own local configuration file stored at .git/config inside the repository directory, which applies only to that specific project. This three-tier configuration hierarchy is an elegant design.

A system administrator can set organisation-wide defaults at the system level. An individual developer can set personal preferences their name, email address, preferred text editor at the global level. And project-specific settings live at the local level. When Git reads a setting, it starts from the most specific level and works outward, with local overriding global, and global overriding system.

User and Group Who Git Runs As, and Why It Matters

Unlike a web server or a database, which run as dedicated system users with restricted permissions, Git does not run as a persistent background service under a separate user account. Each Git command runs as the user who invoked it in the terminal inheriting that user's permissions and nothing more.

This matters for security. Git only has access to the files and directories that the invoking user is permitted to read and write. It cannot access another user's home directory. It cannot write to system directories without explicit permission. It cannot escalate its own privileges. This is the principle of least privilege in practice. When Git is used in a server context, it is common to create a dedicated git system user and configure SSH access so developers authenticate through their own keys rather than a shared password.

Service Management

Git is not a daemon — it does not run continuously in the background as a service. It is a command-line tool that executes a task and exits. When Git is used as a server, it is typically paired with a service that does run as a daemon. The most common approach is running Git over SSH, where the SSH service handles incoming connections:

```
sudo systemctl start sshd
sudo systemctl stop sshd
sudo systemctl restart sshd
sudo systemctl status sshd
```

Alternatively, git daemon is a lightweight built-in server that Git provides for serving repositories over the Git protocol:

```
sudo systemctl start git-daemon
sudo systemctl status git-daemon
```

Update Model - How Security Patches Reach Your Machine

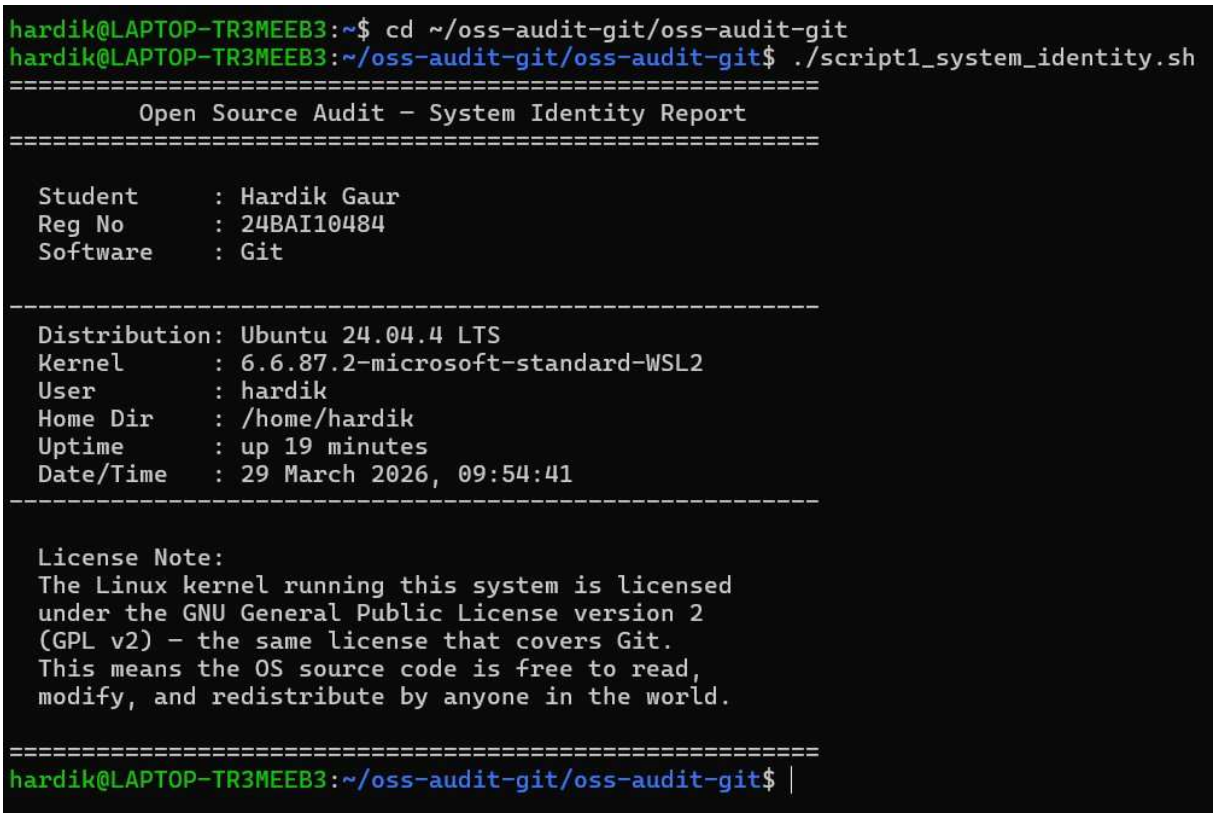
One of the most important advantages of installing software through a Linux package manager is the update model it provides. When a security vulnerability is discovered in Git, the fix is developed, reviewed, and tested in the open Git repository. Once ready, a new version is released and the source code including the security fix is made fully public. Distribution maintainers then package the new version and push it to their repositories. Users receive the update through their package manager:

```
sudo apt-get update
sudo apt-get upgrade git
```

Because Git's source code is public, the security fix can be inspected by anyone in the world. Security researchers can verify that the fix actually addresses the vulnerability. This transparency is itself a security property — it is much harder to quietly introduce a backdoor or leave a known vulnerability unfixed when millions of people can read every line of code. Contrast this with the proprietary model, where a vendor may know about a vulnerability for weeks before patching it and may never disclose the details of what was fixed.

Script Output Screenshots

The following screenshots show all five shell scripts running on Ubuntu 24.04.4 LTS under WSL2, demonstrating the Linux footprint of Git in practice.



```
hardik@LAPTOP-TR3MEEB3:~$ cd ~/oss-audit-git/oss-audit-git
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script1_system_identity.sh
==========================================================
         Open Source Audit - System Identity Report
==========================================================

  Student     : Hardik Gaur
  Reg No      : 24BAI10484
  Software    : Git

----------------------------------------------------------
  Distribution: Ubuntu 24.04.4 LTS
  Kernel      : 6.6.87.2-microsoft-standard-WSL2
  User        : hardik
  Home Dir    : /home/hardik
  Uptime      : up 19 minutes
  Date/Time   : 29 March 2026, 09:54:41
----------------------------------------------------------

  License Note:
  The Linux kernel running this system is licensed
  under the GNU General Public License version 2
  (GPL v2) - the same license that covers Git.
  This means the OS source code is free to read,
  modify, and redistribute by anyone in the world.

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$
```

Figure 1 — Script 1: System Identity Report running on Ubuntu 24.04.4 LTS

```
==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script2_package_inspector.sh
==========================================================
         FOSS Package Inspector
==========================================================

  Inspecting package: git

  Status  : INSTALLED
  Version : 1:2.43.0-1ubuntu7.3

  Binary location:
/usr/bin/git

  License info:
License: GPL-2
License: BSD-3-clause
License: Zlib
License: LGPL-2.1+
License: EDL-1.0

----------------------------------------------------------
  Philosophy note:

  Git: Born from a licensing dispute in 2005, Git proves
  that the best response to proprietary failure is to
  build something free and give it to the world.

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ |
```

Figure 2 — Script 2: FOSS Package Inspector confirming Git v2.43.0 installed under GPL-2

```
==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script3_disk_auditor.sh
==========================================================
         Disk and Permission Auditor
==========================================================

  Auditing standard Linux system directories...

  Directory            Permissions / Owner       Size
  ----------------------------------------------------------
  /etc                 drwxr-xr-x root root      4.3M
  /var/log             drwxrwxr-x root syslog    33M
  /home                drwxr-xr-x root root      104K
  /usr/bin             drwxr-xr-x root root      95M
  /tmp                 drwxrwxrwt root root      24K
  /var                 drwxr-xr-x root root      181M
  /usr/share           drwxr-xr-x root root      273M

----------------------------------------------------------
  Git-specific directory check:

  System config  : /etc/gitconfig (not present - default)

  User config    : /home/hardik/.gitconfig (not configured yet)

  Git binary     : /usr/bin/git
  Permissions    : -rwxr-xr-x root root

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ |
```

Figure 3 — Script 3: Disk and Permission Auditor showing key Linux directories and Git binary permissions

```
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script4_log_analyzer.sh test.log error
=========================================================
         Log File Analyzer
=========================================================

  Log file : test.log
  Keyword  : error

  Result: Keyword 'error' found 2 time(s) in test.log

---------------------------------------------------------
  Last 5 lines containing 'error':

  error: disk quota exceeded
  error: failed to write file

=========================================================
```

Figure 4 — Script 4: Log File Analyzer finding 2 occurrences of the keyword 'error' in test.log

```
=========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script5_manifesto.sh
=========================================================
         Open Source Manifesto Generator
=========================================================

  Answer three questions to generate your manifesto.
  There are no wrong answers — this is your statement.

  1. Name one open-source tool you use every day: Git

  2. In one word, what does 'freedom' mean to you? Choice

  3. Name one thing you would build and share freely: A free-for-all learning platform

  Manifesto saved to: manifesto_hardik.txt

  Here is your manifesto:

=========================================================
  My Open Source Manifesto
  Generated on: 29 March 2026
=========================================================

  Every day, I rely on Git — a tool I did not pay for,
  did not have to ask permission to use, and could modify
  if I ever needed to. That is not an accident. It is the
  result of a deliberate choice made by its creators to
  share their work freely with the world.

  To me, freedom in software means Choice. Not freedom
  as a marketing word, but as a legal guarantee — the
  right to run, study, change, and share. The difference
  between software that respects this and software that
  does not is the difference between a tool you own and
  a service you are permitted to use until further notice.

  If I could build one thing and give it away, it would
  be A free-for-all learning platform. Not because I would not want credit, but
  because the most durable things in software — Linux,
  Git, the web itself — were built by people who cared
  more about the problem than about owning the solution.

  That is what open source means to me.

=========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ |
```

Figure 5 — Script 5: Open Source Manifesto Generator producing a personalised manifesto

Part C — The FOSS Ecosystem Around Git

No piece of software exists in isolation. Every tool has dependencies it relies on, a community that sustains it, and a legacy in the tools it inspired or enabled. Git's ecosystem is one of the largest and most consequential in the entire open-source world. Understanding it means understanding not just Git itself, but the entire infrastructure that modern software development is built on.

What other open-source tools does Git depend on?

Git is built on top of several foundational open-source libraries, each of which is a significant project in its own right. This layering of open-source tools on top of other open-source tools is one of the defining characteristics of the FOSS ecosystem — no project starts from zero. The core libraries that Git depends on include: zlib for data compression, OpenSSL for encrypted communication over HTTPS, libcurl for data transfer over network protocols, libexpat for parsing XML data, and libiconv for handling different character encodings across operating systems.

Zlib is worth understanding in particular because it connects directly to one of Git's most important design decisions. Every object that Git stores every file, every commit, every tree is compressed using zlib before being written to disk. This is how Git keeps repository sizes manageable even when tracking thousands of files across thousands of commits. The compression happens invisibly, and it depends entirely on zlib, a library maintained as free software since 1995.

OpenSSL is the library that makes git push and git pull over HTTPS secure. When the Heartbleed vulnerability was discovered in OpenSSL in 2014, the entire open-source ecosystem including Git had to respond rapidly. The transparency of open source made that response possible. Libcurl handles all of Git's network communication, and the C compiler typically GCC, the GNU Compiler Collection is an implicit dependency, as Git is written in C.

What has Git enabled and inspired?

Git did not just solve the version control problem for the Linux kernel. It created an entirely new category of infrastructure and inspired a generation of tools that now sit at the centre of the global software industry.

The most significant thing Git enabled is GitHub, launched in 2008 by Tom Preston-Werner, Chris Wanstrath, and PJ Hyett. GitHub took Git's distributed model and added a web interface, social features, issue tracking, pull requests, and a public forum for sharing open-source code. It is now the world's largest host of open-source projects. In 2018, Microsoft acquired GitHub for seven and a half billion dollars an extraordinary valuation for a platform built entirely on top of a free, GPL-licensed tool that one engineer wrote in ten days.

GitLab emerged in 2011 as an alternative to GitHub, placing greater emphasis on integrated DevOps capabilities, built-in CI/CD pipelines, and the option for organisations to self-host their own instance. GitLab follows an open-core model its Community Edition is open source, while its Enterprise Edition includes proprietary features. This model reflects a recurring tension in the open-source ecosystem: how

to build a commercially sustainable business while honouring the community values that made the product possible.
Beyond GitHub and GitLab, Git has inspired a wide range of tools: Gitea and its community fork Forgejo are lightweight, self-hostable Git servers for teams that want full control over their infrastructure. Bitbucket, owned by Atlassian, provides Git hosting with deep integration into Jira and Confluence. Every one of these platforms is, at its core, a layer of tooling built on top of Git's open protocol and data model.

Git has also given rise to entirely new development workflows. The pull request model where a developer proposes a change, a reviewer examines it, automated tests run, and a maintainer merges it is now the standard model for software collaboration worldwide, in both open-source and commercial contexts. This workflow was made possible by Git's branching and merging model.

Where does Git fit in the LAMP stack and the web?

LAMP stands for Linux, Apache, MySQL, and PHP the four open-source components that together powered the first generation of dynamic websites. Git is not part of the original LAMP stack, but its relationship to that stack is intimate and essential.

Git is the tool that gets LAMP stack code onto the server in the first place. A developer writes PHP code on their local machine, commits it to a Git repository, and pushes it to a server running Linux, where Apache serves it to the world. In modern deployment workflows, this push to Git automatically triggers a pipeline that tests the code, builds it, and deploys it to the server without any manual intervention. The entire discipline of DevOps the practice of automating the path from code to running software is built around Git as its foundational input.

Git also plays a specific role in the security and integrity of web applications. Because every change to a codebase is recorded in Git with a timestamp, an author, and a cryptographic hash, it is possible to audit exactly what changed, when, and who approved it. This audit trail is a requirement in many regulated industries financial services, healthcare, government where software changes must be traceable for compliance purposes.

Is there a community? Who governs Git?

Git's governance model is deliberately minimal. There is no foundation, no board of directors, no formal voting structure. The project is governed by a benevolent maintainer model, with Junio Hamano at its centre. All development discussion happens on the public Git mailing list at git@vger.kernel.org open to anyone, no subscription required.

Interestingly, Git does not accept contributions through pull requests on GitHub, even though its source code is mirrored there. Instead, contributors format their changes as patch files and email them to the mailing list for review. This patch-by-email workflow is the same model that Linus used to manage the Linux kernel before Git existed, and it remains Git's contribution model to this day. Every discussion about every change is permanently recorded in a public mailing list archive that anyone can search. Beyond the mailing list, the Git community is reachable through the #git channel on the Libera Chat IRC server, where knowledgeable developers are typically online and willing to help.

Part D — Open Source vs Proprietary: Git vs Perforce Helix Core

Every technology choice is also a values choice. When an organisation selects a version control system, it is not simply choosing a tool it is choosing who controls its infrastructure, who can audit it, who can fix it, and what happens when the vendor changes its pricing, its terms, or its priorities. This section examines Git against its closest proprietary alternative Perforce Helix Core across five dimensions, then offers a final verdict.

Why Perforce Helix Core?

The most honest proprietary comparison for Git is Perforce Helix Core, commonly called P4. Perforce had significant traction at enterprise-scale companies notably Google before Git's dominance became total. It remains the standard version control system in the video game industry, used by studios at companies like Epic Games, EA, and Ubisoft, precisely because it handles large binary files textures, audio assets, 3D models better than Git does. It is a serious, mature, commercially supported tool with genuine strengths, and comparing Git against it honestly requires acknowledging those strengths.

Comparison Table

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Cost	Completely free — zero licensing cost for any team of any size	Commercial licensing — costs scale with number of users; enterprise deployments run to tens of thousands of dollars annually
Security	Source code is fully public — any researcher can inspect it, find vulnerabilities, and verify fixes	Source code is proprietary — security depends entirely on Perforce's internal processes; users cannot independently verify claims
Support & Reliability	Community-driven — support through mailing lists, IRC, Stack Overflow; no guaranteed response time without a paid third party	Dedicated commercial support with SLAs — guaranteed response times, direct access to engineers, formal escalation paths
Freedom to Modify	Complete freedom — any organisation can fork Git, modify it, and deploy a custom version; GPL v2 requires sharing changes if distributed	No freedom to modify — source code not available; users are entirely dependent on Perforce's roadmap and release schedule
Community vs Corporate Control	Governed by Junio Hamano and a global volunteer community — decisions made publicly on a mailing list; no single company can change the direction unilaterally	Controlled entirely by Perforce Software Inc. — the company can change pricing, terms, or discontinue the product at any time
Large Binary File Handling	Weak natively — repositories can bloat significantly with large binary assets; Git LFS partially mitigates this	Excellent — specifically optimised for large binary files, making it the standard choice in game development and media production

Dimension	Git (Open Source)	Perforce Helix Core (Proprietary)
Branching & Merging	Outstanding — Git's branching model is lightweight, fast, and the foundation of workflows like GitFlow and trunk-based development	Capable but heavier — Perforce supports branching but it is more administratively complex and less fluid than Git
Offline Work	Full offline capability — every developer has a complete copy of the entire repository history locally	Limited offline capability — centralised, meaning developers must be connected to the server to commit changes and access history
Developer Adoption	Used by approximately 94% of developers worldwide — the de facto industry standard	Niche but dominant in specific industries — game development, embedded systems, and some large enterprises

Written Analysis

Cost is perhaps the most immediately tangible difference. Git costs nothing. Any student, any startup, any organisation anywhere in the world can download and use Git today with no licence fee, no subscription, and no vendor relationship. Perforce, by contrast, is a commercial product with licensing costs that are substantial for large teams and not publicly listed. This cost asymmetry has been one of the most significant drivers of Git's global dominance when one option is free and the other costs tens of thousands of dollars annually, the free option wins in most contexts by default.

On security, the open-source model provides something that no proprietary vendor can match: independent verifiability. Git's source code is publicly available, which means security researchers, universities, and individual developers can examine every line for vulnerabilities. When a flaw is found, the fix is developed in public, reviewed by the community, and released transparently. With Perforce, users are entirely dependent on the company's internal security processes. In 2023, Perforce disclosed a critical vulnerability in Helix Core that allowed unauthenticated remote code execution. The fix was proprietary, and users had no way to audit whether it was complete.

Support and reliability represent the one area where the proprietary model offers a genuine, honest advantage. Commercial support from Perforce comes with service level agreements contractual commitments about response times and resolution targets. For an enterprise organisation running critical infrastructure, this matters. Git's community support is excellent in quality but has no SLAs. GitHub Enterprise and GitLab offer commercial support tiers for Git-based workflows, which partially addresses this gap, but the underlying tool itself has no vendor standing behind it with a contract.

Freedom to modify is where the philosophical difference between open and proprietary software becomes most concrete. Any organisation using Git can take its source code, modify it, and deploy a custom version tailored to their specific needs. Google has done exactly this building internal tooling on top of Git's foundation. No such option exists with Perforce. Users are entirely dependent on Perforce's engineering roadmap. Community versus corporate control returns us directly to the story in Part A. The Linux

kernel community had no control over BitKeeper. When BitMover decided to change its terms, the community was helpless. Git was built specifically to prevent that situation from ever recurring. Because Git is distributed, every developer already has a full copy of the entire repository history on their local machine. Even if every Git server in the world were shut down simultaneously, nothing would be lost. With Perforce, the data lives on a centralised server that the company controls.

Overall Verdict

Would I recommend Git for a real-world deployment? Yes, without hesitation, for the vast majority of use cases. For any team writing software whether a solo developer, a startup, or a large engineering organisation Git is not merely adequate. It is the industry standard, used by 94% of developers worldwide, supported by a global ecosystem of tooling, and governed by a community that has maintained it reliably for twenty years. Its cost is zero, its security model is transparent, its flexibility is unmatched, and no company can ever revoke your right to use it.

The one honest caveat is this: for teams working primarily with large binary assets game studios, film production houses, organisations managing gigabytes of non-text files Git's limitations with binary content are real and should not be dismissed. In those specific contexts, Perforce's centralised model and optimised binary handling represent genuine advantages that Git's architecture cannot match without workarounds.

Would I contribute to Git? Yes. Contribution to an open-source project of Git's importance is one of the most meaningful things a developer can do. Every patch submitted, every bug reported, every documentation improvement made is a contribution to infrastructure that millions of people depend on daily. The contribution process is open to anyone a mailing list, a patch file, and a willingness to receive detailed technical feedback. That openness, and the responsibility it implies, is precisely what makes open source worth defending.

Shell Scripts

The following five shell scripts demonstrate practical Linux skills and connect the philosophical themes of this report to real command-line usage. Each script is documented below with: the complete script code, a screenshot of its output, and an explanation of the concepts it demonstrates.

Script 1 — System Identity Report

This script introduces the Linux system like a welcome screen. It uses variables to store system information retrieved through command substitution — the $() syntax that runs a command and captures its output as a string. The uname -r command retrieves the kernel version, whoami returns the current logged-in user, uptime -p returns human-readable system uptime, date returns the current date and time, and lsb_release -d retrieves the distribution name. All values are stored in named variables and printed using echo with formatted spacing. The script demonstrates the core shell scripting concepts of variable assignment, command substitution, and formatted output.

```
# script1_system_identity.sh
#!/bin/bash
# Script 1: System Identity Report
# Author: Hardik Gaur | Reg No: 24BAI10484
# Course: Open Source Software

STUDENT_NAME="Hardik Gaur"
REG_NO="24BAI10484"
SOFTWARE_CHOICE="Git"

KERNEL=$(uname -r)           # Kernel version
USER_NAME=$(whoami)          # Currently logged-in user
HOME_DIR=$HOME               # Home directory
UPTIME=$(uptime -p)          # Human-readable uptime
DATETIME=$(date '+%d %B %Y, %H:%M:%S')
DISTRO=$(lsb_release -d | cut -f2)

echo "========================================="
echo "  Open Source Audit — System Identity"
echo "========================================="
echo "  Student  : $STUDENT_NAME"
echo "  Kernel   : $KERNEL"
echo "  User     : $USER_NAME"
echo "  Uptime   : $UPTIME"
echo "  Date     : $DATETIME"
echo "  License  : GPL v2 (Linux & Git)"
```

```
hardik@LAPTOP-TR3MEEB3:~$ cd ~/oss-audit-git/oss-audit-git
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script1_system_identity.sh
==========================================================
         Open Source Audit — System Identity Report
==========================================================

  Student      : Hardik Gaur
  Reg No       : 24BAI10484
  Software     : Git

----------------------------------------------------------
  Distribution: Ubuntu 24.04.4 LTS
  Kernel       : 6.6.87.2-microsoft-standard-WSL2
  User         : hardik
  Home Dir     : /home/hardik
  Uptime       : up 19 minutes
  Date/Time    : 29 March 2026, 09:54:41
----------------------------------------------------------

  License Note:
  The Linux kernel running this system is licensed
  under the GNU General Public License version 2
  (GPL v2) — the same license that covers Git.
  This means the OS source code is free to read,
  modify, and redistribute by anyone in the world.

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$
```

Figure 1 — Script 1 output: System Identity Report on Ubuntu 24.04.4 LTS, WSL2

Script 2 — FOSS Package Inspector

This script checks whether Git is installed on the system, retrieves its version and package metadata, and uses a case statement to print a philosophy note about each recognised package. The if-then-else block uses dpkg -l on Debian-based systems or rpm -q on RPM-based systems to detect installation status. The pipe with grep filters output to show only the relevant fields — version, license, and summary. The case statement matches the package name against known values and prints a relevant philosophical note. This script demonstrates conditional logic, package management commands, pipes, and pattern matching.

```
# script2_package_inspector.sh
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Hardik Gaur | Reg No: 24BAI10484

PACKAGE="git"

if command -v dpkg &>/dev/null; then
    PKG_MANAGER="dpkg"
elif command -v rpm &>/dev/null; then
    PKG_MANAGER="rpm"
fi

if [ "$PKG_MANAGER" = "dpkg" ]; then
    if dpkg -l "$PACKAGE" &>/dev/null; then
        INSTALLED="yes"
        VERSION=$(dpkg -l "$PACKAGE" | grep "^ii" | awk '{print $3}')
```

```
    fi
fi

case $PACKAGE in
    git)    echo "Git: Born from a licensing dispute in 2005" ;;
    httpd)  echo "Apache: built the open internet" ;;
    firefox) echo "Firefox: fighting for the open web" ;;
esac
```

```
==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script2_package_inspector.sh
==========================================================
         FOSS Package Inspector
==========================================================

  Inspecting package: git

  Status  : INSTALLED
  Version : 1:2.43.0-1ubuntu7.3

  Binary location:
/usr/bin/git

  License info:
License: GPL-2
License: BSD-3-clause
License: Zlib
License: LGPL-2.1+
License: EDL-1.0

----------------------------------------------------------
  Philosophy note:

  Git: Born from a licensing dispute in 2005, Git proves
  that the best response to proprietary failure is to
  build something free and give it to the world.

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ |
```

Figure 2 — Script 2 output: Git v2.43.0 detected as INSTALLED under GPL-2

Script 3 — Disk and Permission Auditor

This script loops through a predefined list of important Linux system directories and reports the permissions, owner, group, and disk usage of each one. The for loop iterates over the DIRS array, and inside each iteration an if block checks whether the directory exists before attempting to inspect it. The ls -ld command retrieves the permission string and ownership details, and awk extracts specific fields from that output. The du -sh command measures disk usage and cut isolates the size value. A final section specifically checks for Git's configuration directory. This script demonstrates arrays, for loops, conditional checks, and field extraction with awk and cut.

```
# script3_disk_auditor.sh
#!/bin/bash
# Script 3: Disk and Permission Auditor
```

```
# Author: Hardik Gaur | Reg No: 24BAI10484

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/var" "/usr/share")

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "  %-20s %-25s %-10s\n" "$DIR" "$PERMS" "$SIZE"
    else
        printf "  %-20s %s\n" "$DIR" "does not exist"
    fi
done
```

```
==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script3_disk_auditor.sh
==========================================================
         Disk and Permission Auditor
==========================================================

  Auditing standard Linux system directories...

  Directory            Permissions / Owner       Size
  ------------------------------------------------------------
  /etc                 drwxr-xr-x root root      4.3M
  /var/log             drwxrwxr-x root syslog    33M
  /home                drwxr-xr-x root root      104K
  /usr/bin             drwxr-xr-x root root      95M
  /tmp                 drwxrwxrwt root root      24K
  /var                 drwxr-xr-x root root      181M
  /usr/share           drwxr-xr-x root root      273M

----------------------------------------------------------
  Git-specific directory check:

  System config  : /etc/gitconfig (not present - default)

  User config    : /home/hardik/.gitconfig (not configured yet)

  Git binary     : /usr/bin/git
  Permissions    : -rwxr-xr-x root root

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$
```

Figure 3 — Script 3 output: Disk and Permission Auditor showing directory sizes and permissions

Script 4 — Log File Analyzer

This script reads a log file line by line using a while read loop and counts how many lines contain a specified keyword. The log file path and keyword are passed as command-line arguments using $1 and $2, with a default keyword of 'error' applied using the ${2:-'error'} syntax when no second argument is provided. Inside the loop, an if-then block uses grep -iq to perform a case-insensitive match against the keyword, and a counter variable is incremented each time a match is found. After the loop, a retry mechanism checks if the count was zero and re-runs the search with a broader term. The last five matching lines are printed using a combination of grep and tail. This

script demonstrates while read loops, command-line arguments, default values, counters, and the grep and tail commands.

```
# script4_log_analyzer.sh
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Hardik Gaur | Reg No: 24BAI10484
# Usage: ./script4_log_analyzer.sh /var/log/syslog error

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default to 'error' if not provided
COUNT=0

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s)"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

```
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script4_log_analyzer.sh test.log error
==========================================================
         Log File Analyzer
==========================================================

  Log file : test.log
  Keyword  : error

  Result: Keyword 'error' found 2 time(s) in test.log

----------------------------------------------------------
  Last 5 lines containing 'error':

  error: disk quota exceeded
  error: failed to write file

==========================================================
```

Figure 4 — Script 4 output: Log File Analyzer finding 2 occurrences of 'error' in test.log

Script 5 — The Open Source Manifesto Generator

This script interactively asks the user three questions using the read command with the -p flag to display a prompt inline. The answers are stored in three variables — TOOL, FREEDOM, and BUILD — which are then concatenated using string interpolation inside echo statements to compose a personalised paragraph. The output is written to a uniquely named .txt file using the > redirect operator, with the filename dynamically generated by incorporating the current username from $(whoami). The date command with a format string generates a formatted date stamp. The cat command then reads and displays the saved file. A comment inside the script demonstrates the concept of aliases. This script demonstrates interactive input, string concatenation, file writing with redirection, and the date and cat commands.

```
# script5_manifesto.sh
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Hardik Gaur | Reg No: 24BAI10484
```

```
# Alias concept (demonstrated via comment):
# alias generate='./script5_manifesto.sh'

read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does freedom mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "My Open Source Manifesto" > "$OUTPUT"
echo "Generated on: $DATE" >> "$OUTPUT"
echo "I rely on $TOOL every day..." >> "$OUTPUT"
echo "To me, freedom means $FREEDOM." >> "$OUTPUT"
echo "I would build $BUILD and share it freely." >> "$OUTPUT"

cat "$OUTPUT"
```

```
==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ ./script5_manifesto.sh
==========================================================
         Open Source Manifesto Generator
==========================================================

  Answer three questions to generate your manifesto.
  There are no wrong answers — this is your statement.

  1. Name one open-source tool you use every day: Git

  2. In one word, what does 'freedom' mean to you? Choice

  3. Name one thing you would build and share freely: A free-for-all learning platform

  Manifesto saved to: manifesto_hardik.txt

  Here is your manifesto:

==========================================================
  My Open Source Manifesto
  Generated on: 29 March 2026
==========================================================

  Every day, I rely on Git — a tool I did not pay for,
  did not have to ask permission to use, and could modify
  if I ever needed to. That is not an accident. It is the
  result of a deliberate choice made by its creators to
  share their work freely with the world.

  To me, freedom in software means Choice. Not freedom
  as a marketing word, but as a legal guarantee — the
  right to run, study, change, and share. The difference
  between software that respects this and software that
  does not is the difference between a tool you own and
  a service you are permitted to use until further notice.

  If I could build one thing and give it away, it would
  be A free-for-all learning platform. Not because I would not want credit, but
  because the most durable things in software — Linux,
  Git, the web itself — were built by people who cared
  more about the problem than about owning the solution.

  That is what open source means to me.

==========================================================
hardik@LAPTOP-TR3MEEB3:~/oss-audit-git/oss-audit-git$ |
```

Figure 5 — Script 5 output: Personalised Open Source Manifesto saved to manifesto_hardik.txt